

Dinâmica "A Arte do Early Return"

1. Debater para pensar (Perguntas de Discussão)

Pergunta 1: Qual código é mais fácil de explicar em voz alta para um colega?

- **Resposta Esperada:** A "Opção Direta" (Opção B).
- **Motivo:** Ela pode ser lida como um manual de regras linear e simples: *"Se o usuário não estiver ativo, não tem desconto. Se não tiver produto, não tem desconto. Se o cupom for válido, é 15%. Caso contrário, 5%"*. Na "Opção Pirâmide", o cérebro precisa guardar o contexto do primeiro `if` enquanto avalia o segundo e o terceiro, gerando confusão mental durante a explicação.

Pergunta 2: Se precisarmos adicionar uma nova regra agora (ex: `if (usuario.isBlackFriday)`), qual código fica mais fácil de modificar sem quebrar o que já existe?

- **Resposta Esperada:** A "Opção Direta" (Opção B).
- **Motivo:** Para adicionar uma nova regra na "Opção Direta", basta inserir uma nova linha (uma nova cláusula de guarda ou condição direta), sem mexer na estrutura do restante do código. Na "Opção Pirâmide", uma nova regra exigiria envolver todo o código existente dentro de um novo bloco `if`, aumentando ainda mais a indentação e o risco de errar o fechamento de alguma chave (`}`).

Pergunta 3: Por que a "Opção Pirâmide" pode ser considerada perigosa em projetos grandes?

- **Resposta Esperada:** Ela cria o que chamamos de "Arrow Anti-Pattern" (ou código Hadouken).
- **Motivo:** A complexidade ciclomática cresce rapidamente. A separação visual entre o `if` e o seu respectivo `else` fica tão distante que o programador precisa rolar a tela para entender a qual condição aquele erro pertence. Isso aumenta drasticamente a carga cognitiva, tornando o código frágil, difícil de testar e propenso a bugs ocultos.

2. O Relatório de Refatoração (Entrega da Equipe)

As respostas das equipes devem conter elementos parecidos com a estrutura abaixo:

A Escolha Técnica:

Será adotada a **"Opção Direta" (Early Return)** como padrão no projeto.

Justificativa "Clean Code" (3 pontos obrigatórios):

Os alunos devem listar pelo menos três dos seguintes argumentos técnicos:

1. **Redução da Indentação:** Mantém o código plano (flat), evitando aninhamentos profundos que dificultam a leitura.
2. **Redução da Carga Cognitiva:** Libera espaço mental. Quando a função passa por uma cláusula de guarda (ex: falha de usuário inativo), o programador não precisa mais se preocupar com aquele cenário no resto da leitura.
3. **Destaque para o "Caminho Feliz" (Happy Path):** As validações de erro e exceções são resolvidas logo no início, deixando a lógica principal e de sucesso clara e livre de poluição visual.
4. **Fail Fast (Falhe Rápido):** O sistema não processa variáveis ou lógicas desnecessárias se uma condição primária já impede o sucesso da operação, economizando processamento (mesmo que micro).

O Teste de Stress:

- **Como deve ser a resposta:** Se a regra de usuário VIP for adicionada, na "Opção Direta" o código não sofrerá estresse estrutural. A nova condição será adicionada de forma limpa e sequencial logo

após as cláusulas de guarda. O código não precisa ser re-indentado e as regras antigas permanecem intactas.

- **Exemplo prático (Opcional para a equipe incluir):**

JavaScript

```
function calcularDesconto(usuario, produto, cupom) {  
  if (!usuario.estaAtivo) return 0;  
  if (produto.quantidade <= 0) return 0;  
  
  // Nova regra VIP adicionada sem estresse estrutural:  
  if (usuario.isVip) return 0.20;  
  
  if (cupom.isValid) return 0.15;  
  
  return 0.05;  
}
```

/**

* Arquivo: utils.js

* Gabarito dos Exercícios de Fixação

*/

// =====

// 1. Minhas Primeiras Variáveis

// =====

// const é usado para variáveis que não terão seu valor reatribuído (imutáveis)

// let é usado para variáveis que podem mudar de valor ao longo do tempo (mutáveis)

const nomeProduto = "Teclado Mecânico";

let quantidadeEstoque = 50;

const produtoAtivo = true;

// =====

// 2. Saudação ao Cliente

// =====

function saudarCliente(nome) {

// Utilizando Template Literals (crases) para facilitar a interpolação da variável

return `Olá, \${nome}! Bem-vindo à nossa loja.`;

}

// =====

// 3. Formatar para Moeda

// =====

function formatarMoedaBRL(valor) {

// O método .toFixed(2) garante que o número terá exatamente 2 casas decimais.

return `R\$ \${valor.toFixed(2)}`;

}

```
// =====
// 4. Desconto de Funcionário
// =====

function calcularDesconto(precoOriginal, isFuncionario) {
    // Aplicando o conceito de Early Return (Saída Antecipada)
    if (isFuncionario) {
        return precoOriginal * 0.70; // Desconto de 30%
    }

    return precoOriginal;
}

// =====
// 5. Montando o Produto
// =====

const produto = {
    id: 101,
    nome: "Monitor 24 Polegadas",
    preco: 850.00,
    categorias: ["Informática", "Monitores", "Escritório"] // Array de strings
};

// =====
// 6. Validador de Senha Forte
// =====

function validarSenha(senha) {
    // Retorna true apenas se TODAS as condições forem verdadeiras simultaneamente
    return senha.length >= 8 && senha !== "12345678" && senha !== "senha";
}

// =====
// 7. Calculadora Total do Carrinho
// =====

function fecharCarrinho(valorProduto, quantidade, valorFrete) {
    const totalProdutos = valorProduto * quantidade;

    // Se o total dos produtos passar de 200, o frete é zerado
    if (totalProdutos > 200) {
        valorFrete = 0;
    }

    return totalProdutos + valorFrete;
}

// =====
// 8. Validador de CPF (Tamanho)
```

```
// =====
function validarTamanhoCPF(cpf) {
  // .trim() remove os espaços em branco do início e do fim da string
  const cpfLimpo = cpf.trim();

  // Verifica se tem 11 caracteres E se contém apenas números
  // (A expressão regular /\d+$/ testa se a string possui apenas dígitos de 0 a 9)
  return cpfLimpo.length === 11 && /\d+$/.test(cpfLimpo);
}

// =====
// 9. Checagem de Campo Obrigatório
// =====
function validarCampoVazio(campo) {
  // Retorna false se for null, undefined ou uma string vazia/só com espaços
  if (campo === null || campo === undefined || campo.trim() === "") {
    return false;
  }

  return true;
}

// =====
// 10. Resumo da Compra
// =====
function gerarResumo(nomeCliente, totalCompra) {
  // Chamando a função formatarMoedaBRL criada no exercício 3
  const valorFormatado = formatarMoedaBRL(totalCompra);

  return `Cliente: ${nomeCliente}, Total a Pagar: ${valorFormatado}`;
}

// Exportando as funções caso este arquivo seja usado como um módulo no futuro
// module.exports = { saudarCliente, formatarMoedaBRL, calcularDesconto, ... };

```